Technical Report: General Ledger System

K-ERP Accounting Module (Double-Entry General Ledger)

Prepared following ISO/IEC/IEEE 15289 · 29148 · 42010 · 29119-3 · ISO/IEC 25010

Document ID	KERP-ACC-TR-001
Version	1.0
Issue date	12 October 2026
Author	Kongkat Thanalertrungroj
Status	Issued

Document control

Item	Value
Identifier	KERP-ACC-TR-001
Title	Technical Report: General Ledger System — K-ERP Accounting Module (Double-Entry General Ledger)
Information item type	Technical report (combined SRS, architecture description, test documentation and quality evaluation)
Version	1.0
Status	Issued
Issue date	12 October 2026
Author / owner	Kongkat Thanalertrungroj
System of interest	K-ERP Accounting: Accounting-Backend and Accounting-Frontend (latest commits 10 Oct 2026)
Classification	Public (portfolio)
Standards basis	ISO/IEC/IEEE 15289 · 29148 · 42010 · 29119-3 · ISO/IEC 25010

Revision history

Version	Date	Author	Description
1.0	12 October 2026	Kongkat Thanalertrungroj	Initial issue

Review and approval

Role	Name	Date
Prepared by	Kongkat Thanalertrungroj	12 October 2026
Reviewed by		
Approved by		

Contents

1. Introduction **5**
1.1 Purpose 5
1.2 Scope 5
1.3 Intended audience 5
1.4 Definitions and acronyms 5
1.5 Normative references 6
1.6 Document overview 6
2. System overview **7**
2.1 Background 7
2.2 Product perspective 7
2.3 Product functions 8
2.4 User classes 8
2.5 Operating environment 8
2.6 Design constraints 8
2.7 Assumptions and dependencies 8
3. Software requirements specification **10**
3.1 Functional requirements 10
3.2 Business rules 11
3.3 Quality requirements (per ISO/IEC 25010) 11
3.4 External interface requirements 12
4. Architecture description **13**
4.1 Stakeholders and concerns 13
4.2 Viewpoints 13
4.3 Context view 13
4.4 Logical (module) view 13
4.5 Information view 14
4.6 Interaction view: transactional outbox 15
4.7 Lifecycle view: journal entry 15
4.8 Deployment view 15
4.9 Security view 16
4.10 Architecture decision records 16
5. Detailed design **18**
5.1 Posting procedure 18
5.2 Idempotent event booking 18
5.3 Year-end close 18
5.4 Report computation 18
5.5 Error model 18
5.6 Configuration 19
6. Test documentation **20**
6.1 Test plan 20
6.2 Test design 20
6.3 Test completion report 20
6.4 Incident (anomaly) report 21
7. Quality evaluation (ISO/IEC 25010) **22**

7.1 Findings and recommendations 22
8. Requirements traceability matrix 23
9. Limitations, risks and future work 25
9.1 Intentional limitations 25
9.2 Risks 25
9.3 Future work 25
10. Conclusion 26
Appendix A. API reference 27
Appendix B. Data dictionary 28
Appendix C. Default chart of accounts 30
Appendix D. Automatic posting rules 31

1. Introduction

1.1 Purpose

This document gives a complete description of the K-ERP double-entry General Ledger: its requirements, architecture, detailed design, testing and quality evaluation. A reader should be able to understand what the system does, how and why it is designed that way, and how its correctness is demonstrated, without reading the source code.

1.2 Scope

The scope is the Accounting module, made up of Accounting-Backend (REST API) and Accounting-Frontend (web application), together with its interfaces to K-ERPHub (authentication hub) and the Stock & Purchase module (the source of accounting events). The internals of those two systems are described only as far as the interfaces require.

In scope	Out of scope
Chart of accounts, journal entries, accounting periods, year-end close, financial reports, Excel export, event intake from Stock, authentication and authorization	VAT and withholding tax, supplier invoices and payments (AP), receivables (AR), multi-currency, multi-company, budgeting, FIFO/average costing, production deployment

1.3 Intended audience

- Technical reviewers such as interviewers and engineering leads
- Developers who will maintain or extend the system
- Accounting users who need to understand the rules the system enforces

1.4 Definitions and acronyms

Term	Meaning
GL	General Ledger
JE	Journal Entry: a set of at least two debit/credit lines
Double-entry	Every entry must have total debits equal to total credits
Satang	1 THB = 100 satang. All amounts are stored as whole satang.
Posting	Moving an entry from DRAFT to POSTED and assigning its number
Reversal	A new entry with debits and credits swapped that cancels an earlier entry
Period	A calendar month in Bangkok time (UTC+7), e.g. 2026-10
GR/IR	Goods Received / Invoice Received clearing account (2150)
Outbox	Pattern that writes an event in the same transaction as the business data and delivers it later
Idempotency	Repeating the same request any number of times has the same effect as sending it once
JWT	JSON Web Token, a signed authentication token
RBAC	Role-Based Access Control

Term	Meaning
SPA	Single-Page Application
ORM	Object-Relational Mapping; Prisma in this system

1.5 Normative references

Standard	Used for
ISO/IEC/IEEE 15289:2019	Information item format: document control, revision history, structure
ISO/IEC/IEEE 29148:2018	Software requirements specification (Sections 2–3)
ISO/IEC/IEEE 42010:2022	Architecture description: stakeholders, concerns, viewpoints, views, decisions (Section 4)
ISO/IEC/IEEE 29119-3:2021	Test documentation: plan, design, completion report, incidents (Section 6)
ISO/IEC 25010:2023	Product quality model, used for quality requirements and evaluation (Sections 3.3 and 7)

> *Conformance statement: the structure and content of this document follow the standards above. It has not been audited or certified by a third party.*

1.6 Document overview

Section	Content	Standard
2	System overview, users, constraints, assumptions	ISO/IEC/IEEE 29148
3	Functional, business-rule, quality and interface requirements	ISO/IEC/IEEE 29148, ISO/IEC 25010
4	Architecture description and decisions	ISO/IEC/IEEE 42010
5	Detailed design, algorithms, API, configuration	—
6	Test documentation and results	ISO/IEC/IEEE 29119-3
7	Quality evaluation and findings	ISO/IEC 25010
8	Requirements traceability matrix	ISO/IEC/IEEE 29148
9–10	Limitations, risks, future work, conclusion	—
Appendices	API, data dictionary, chart of accounts, posting rules	—

2. System overview

2.1 Background

K-ERP is a modular ERP. Each module is an independent service with its own database, and K-ERPHub is the single point of login. The first module is Stock & Purchase. The Accounting module described here is the second: it keeps a double-entry ledger and books events from the stock module automatically, so that the inventory value in the books matches the stock movements.

2.2 Product perspective

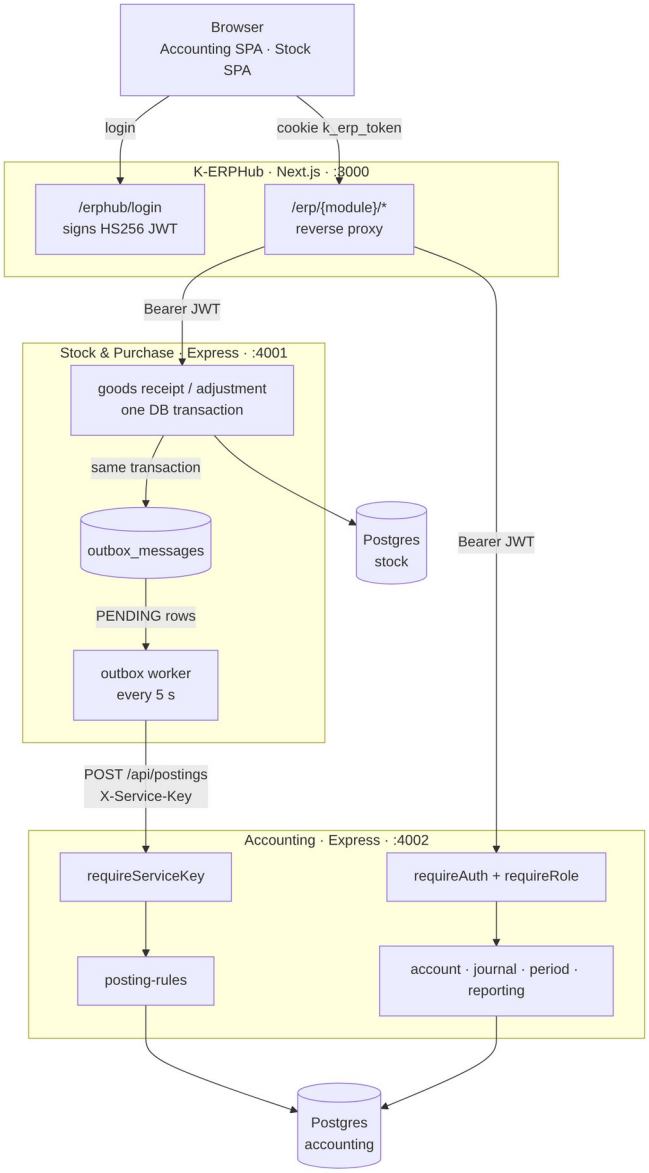


Figure 2.1 System context

Users log in at K-ERPHub, which issues a JWT stored in an httpOnly cookie. K-ERPHub forwards /erp/accounting/* to the Accounting API with the token attached. On the Stock side, events are written to an outbox table in the same transaction as the goods receipt, and a worker delivers them to Accounting through a dedicated endpoint protected by a service key.

2.3 Product functions

- Maintain a hierarchical chart of accounts (group and postable accounts)
- Draft, post and reverse journal entries, checking balance every time
- Control monthly periods, close periods and close the year into retained earnings
- Produce the trial balance, general ledger, income statement and balance sheet, with Excel export
- Receive goods-receipt, stock-adjustment and opening-balance events from Stock and book each exactly once

2.4 User classes

User class	Identified by	Permissions
Module member (viewer)	JWT with "accounting" in allowedModules	Read accounts, entries, periods and all reports
Accountant	JWT role Accountant	Viewer rights plus managing accounts and creating, editing, posting and reversing entries
Admin	JWT role Admin	All Accountant rights plus closing periods and years
Stock system (system:stock)	X-Service-Key header	May call POST /api/postings only

2.5 Operating environment

Component	Detail
Runtime	Node.js 20+, TypeScript (ES modules)
Database	PostgreSQL 16 in Docker (port 5433), one database per module
Browser	Modern browser with ES2020 and BigInt support
Ports	K-ERPHub 3000 · Stock 4001 · Accounting 4002

2.6 Design constraints

- No floating point for money: BigInt satang in code and database
- The module must not have its own login; it only verifies JWTs issued by K-ERPHub
- Periods are based on Bangkok time (UTC+7), not UTC
- Accounting and Stock cannot share a transaction because their databases are separate

2.7 Assumptions and dependencies

- K-ERPHub and this module share the same JWT_SECRET (HS256); tokens expire after 8 hours by default
- Stock sends the computed value in satang with each event; Accounting never calls back into Stock

- The chart of accounts must contain codes 1300, 2150, 3200 and 5200, which the posting rules use

3. Software requirements specification

Each requirement has an identifier used for traceability in Section 8. Priority is M (Must) or S (Should). Verification is T (Test), I (Inspection) or D (Demonstration). Every requirement listed is implemented in the code as of the issue date.

3.1 Functional requirements

ID	Requirement	Prio.	Verif.
FR-01	The system shall maintain a hierarchical chart of accounts of 5 types (asset, liability, equity, revenue, expense); a parent must be a group account.	M	T
FR-02	Once an account has lines, the system shall forbid changes to code, type, group flag and parent, but allow renaming and (de)activation.	M	T
FR-03	The system shall forbid deleting an account that has lines or child accounts.	M	T
FR-04	The system shall allow creating, editing and deleting draft entries; entries from other modules cannot be edited.	M	T
FR-05	The system shall check that an entry has at least 2 lines, each with exactly one positive side, and equal debit and credit totals, both on save and on post.	M	T
FR-06	On posting, the system shall assign a number JE-YYYY-MM-NNNN that is consecutive without gaps or duplicates within the month.	M	T
FR-07	The system shall reverse a posted entry by creating a mirror entry and marking the original REVERSED, at most once, and never reverse a reversal.	M	T
FR-08	The system shall use monthly periods in Bangkok time and create a period as OPEN automatically on first use.	M	T
FR-09	The system shall let an Admin close a period, refusing while drafts remain, and shall reject manual posting or reversal into a closed period.	M	T
FR-10	The system shall let an Admin preview and perform a year-end close that transfers revenue and expenses to retained earnings (3200) and locks all 12 periods, once per year and in year order.	S	T
FR-11	The system shall produce a trial balance as of a date, with a balanced flag.	M	T
FR-12	The system shall show the general ledger for an account with opening, running and closing balances.	M	T
FR-13	The system shall produce an income statement for a date range, excluding the year-end closing entry.	S	T
FR-14	The system shall produce a balance sheet as of any date in which assets equal liabilities plus equity including unclosed net income.	S	T
FR-15	The system shall export the trial balance, income statement and balance sheet to Excel with real numeric cells.	S	T
FR-16	The system shall accept GOODS_RECEIVED, STOCK_ADJUSTED and OPENING_BALANCE events from Stock and turn them into entries according to the posting rules.	M	T
FR-17	The system shall book the same event only once, even if it is received repeatedly or concurrently.	M	T
FR-18	The system shall not reject an event because its period is closed; it shall post it in the next open period and record the actual date.	M	T

ID	Requirement	Prio.	Verif.
FR-19	The system shall authenticate with the K-ERPHub JWT, check access to the "accounting" module and check the role for each function.	M	T
FR-20	The system shall provide GET /api/me so the web UI can hide buttons the user cannot use.	S	T
FR-21	The system shall list entries filtered by status, date range and origin (manual or system).	S	D

3.2 Business rules

ID	Rule
BR-01	Total debits equal total credits in every entry.
BR-02	Each line has exactly one side greater than zero; amounts are never negative.
BR-03	Lines may only use postable (non-group), active accounts.
BR-04	Status may only move DRAFT → POSTED → REVERSED.
BR-05	Posted entries cannot be edited or deleted; errors are corrected only by reversal.
BR-06	Money is stored as whole satang (BigInt) and sent through the API as strings.
BR-07	A goods receipt debits 1300 Inventory and credits 2150 GR/IR, not trade payables.
BR-08	Stock adjustments are keyed on the stock movement id, not on the reason text.
BR-09	Reports count POSTED and REVERSED entries and exclude DRAFT.

3.3 Quality requirements (per ISO/IEC 25010)

ID	Characteristic	Requirement
NFR-01	Functional suitability: correctness	Amounts must be exact to the satang, including values beyond int32 (over about 21.4 million THB).
NFR-02	Reliability: fault tolerance	Goods receipts in Stock must succeed while Accounting is down and be booked later without duplicates.
NFR-03	Security: integrity	Posted entries must be unchangeable even with direct SQL.
NFR-04	Security: authenticity	Every endpoint except /health must check a token or service key; key comparison must be constant-time.
NFR-05	Security: accountability	Creator and poster must come from the token, never from the request body.
NFR-06	Reliability: correctness under concurrency	Concurrent posts and events must not cause duplicate or skipped numbers or double booking.
NFR-07	Maintainability: modularity	Business rules must live in functions without I/O, and posting account codes in one place.
NFR-08	Maintainability: testability	Database-enforced rules must be tested against real PostgreSQL, in a test database separate from development data.
NFR-09	Compatibility: interoperability	Event payloads must carry a schema version, and responses must distinguish retryable from permanent failures.
NFR-10	Interaction capability: user error protection	The entry form must show debit, credit and difference totals live.

ID	Characteristic	Requirement
NFR-11	Flexibility: installability	The system must be installable with Docker, npm scripts and a .env file, without code changes.

3.4 External interface requirements

3.4.1 User interface

Screen	Function
Journal Entries	All entries, filtered by status, date range and origin
New / Edit Journal Entry	Multi-line form with live debit, credit and difference totals
Journal Entry Detail	Detail view, post and reverse buttons, link to the reversal
Chart of Accounts	Add accounts, activate or deactivate, show type and group flag
Trial Balance	Trial balance as of a date, with Excel export
General Ledger	Ledger with running balance
Income Statement	Income statement for a range, with Excel export
Balance Sheet	Balance sheet as of a date, with Excel export
Accounting Periods	Period list, period close, year-end preview and close

3.4.2 Software interfaces

System	Interface
K-ERPHub	Receives forwarded requests with Authorization: Bearer <JWT> carrying sub, username, roles, allowedModules
Stock & Purchase	POST /api/postings with X-Service-Key; body { topic, payload }, payload carries version = 1 and occurredAt
PostgreSQL 16	Through Prisma 6, with raw SQL for row locks, numbering and aggregation

3.4.3 Communication interfaces

HTTP/JSON. Every endpoint is under /api except /health. Excel files are sent as application/vnd.openxmlformats-officedocument.spreadsheetml.sheet.

4. Architecture description

4.1 Stakeholders and concerns

Stakeholder	Concerns	Addressed in
Accountant	Correct, balanced and auditable figures	4.5, 4.7
Admin	Period control, safe year-end close, permissions	4.7, 4.9
Stock & Purchase team	Receiving works while Accounting is down; no double booking	4.6
Auditor	Immutability, knowing who did what	4.5, 4.9
Developer	Clear structure, easy testing, safe change	4.4
Operator	Simple installation and configuration	4.8

4.2 Viewpoints

Viewpoint	Concerns framed	Model kinds
Context	Where the system sits and what it talks to	Context diagram (Fig. 2.1)
Logical / module	How responsibilities are split	Layer diagram + table
Information	What is stored and how integrity is kept	ER diagram
Interaction	How cross-service data is never lost or duplicated	Sequence diagram
Lifecycle	How an entry changes state	State diagram
Deployment	Where it runs and on which ports	Table
Security	Who can access what	Table

4.3 Context view

See Figure 2.1. There are three external actors: users in a browser (through K-ERPHub), the Stock module (through its outbox worker), and PostgreSQL.

4.4 Logical (module) view

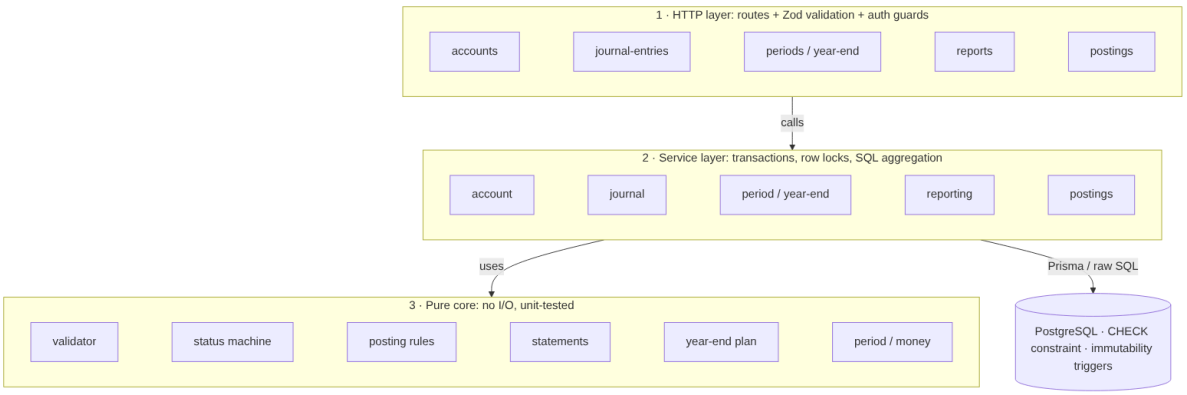


Figure 4.1 Backend layers

Module	Responsibility
auth	JWT verification, role guard, service-key guard, GET /api/me
account	Chart of accounts and structural-change rules
journal	Draft/post/reverse, balance validator (pure), status machine (pure)
period	Lazy period creation, period close, next-open resolution, year-end close (pure plan)
posting-rules	Event to lines (pure), account mapping, idempotent booking
reporting	bigint SQL aggregation, statement builders (pure), Excel
shared	Prisma client, error handler, money/date/period utilities

Dependency rules: other modules go through service functions; for example posting-rules and year-end call createDraftTx and postEntryTx instead of writing journal rows directly, so every path runs the same checks. Functions ending in Tx take the caller's transaction client so multi-step work commits or rolls back as a unit.

4.5 Information view

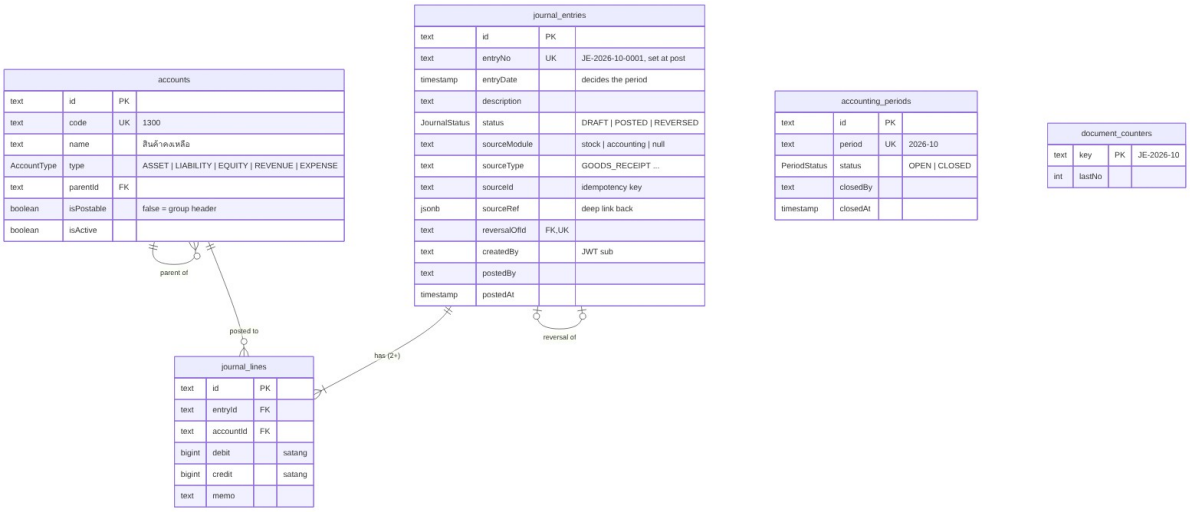


Figure 4.2 Entity-relationship diagram

The database has 5 tables: accounts, journal_entries, journal_lines, accounting_periods and document_counters. Integrity is kept by (1) CHECK one_side_only on journal_lines, (2) triggers guard_posted_entry and guard_posted_lines that forbid changing or deleting non-DRAFT entries except the POSTED → REVERSED status flip, and (3) unique keys on (sourceModule, sourceType, sourceId) and on reversalOfId. Column details are in Appendix B.

4.6 Interaction view: transactional outbox

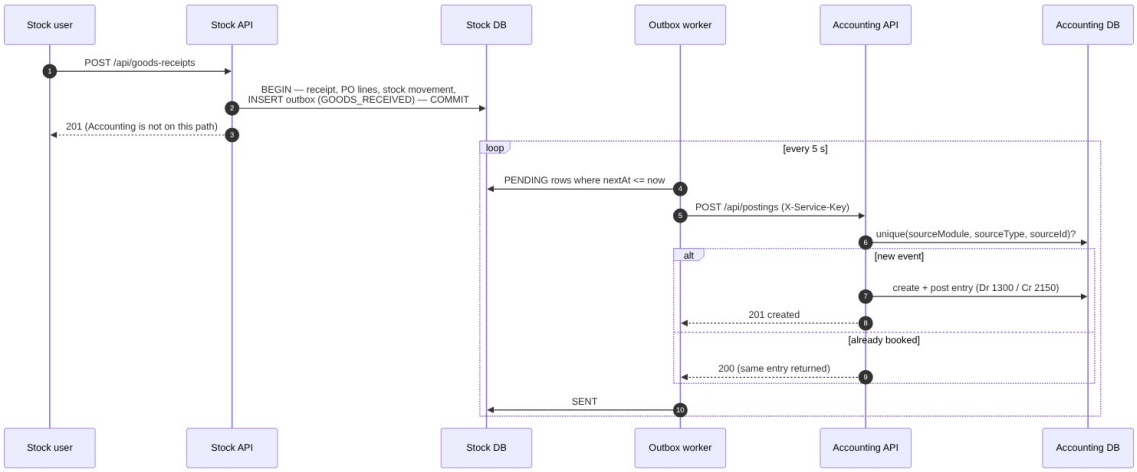


Figure 4.3 Delivering a goods-receipt event to the ledger

Accounting response	Meaning	Worker action
2xx	Booked (new or replay)	SENT
network error, timeout, 408, 429, 5xx	Temporary	Retry after 5 s, 10 s, 20 s … up to 15 min, at most 8 attempts
Other 4xx	Permanent	FAILED at once, error kept; an Admin can retry after fixing the cause

4.7 Lifecycle view: journal entry

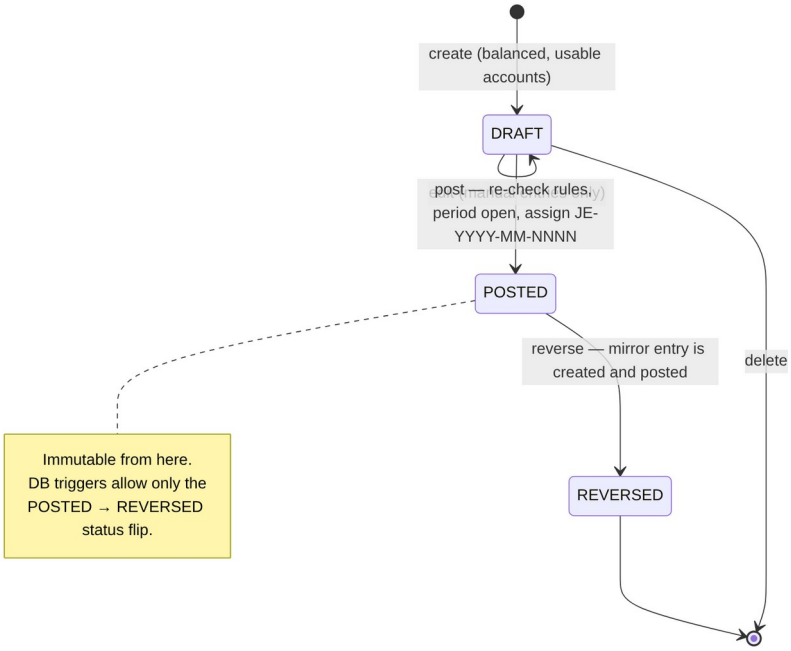


Figure 4.4 Journal entry states

4.8 Deployment view

Service	Port	Notes
K-ERPHub (Next.js 16)	3000	User entry point

Service	Port	Notes
Stock & Purchase API + SPA	4001	Outbox worker runs in the same process every 5 s
Accounting API + SPA	4002	Serves the built SPA from ../Accounting-Frontend/dist under /erp/accounting/
PostgreSQL 16	5433	One container, a database per module

4.9 Security view

Aspect	Design
Authentication	HS256 JWT from K-ERPHub, 8-hour default expiry, verified locally with the shared JWT_SECRET
Module access	"accounting" must be in allowedModules, otherwise 403 even for Admin
Roles	Read: module member · Write: Accountant, Admin · Period and year close: Admin
Service-to-service	X-Service-Key compared in constant time (SHA-256 + timingSafeEqual), valid only on /api/postings; fails closed if not configured
Audit	createdBy and postedBy come from the token sub; system events are recorded as system:stock
Web UI	Never handles tokens; uses the httpOnly cookie via the proxy. Hiding buttons is UX only, enforcement is in the API

4.10 Architecture decision records

ID	Decision	Rationale	Alternatives / consequences
ADR-01	Separate database per module, events via a transactional outbox	Calling the API directly after a receipt loses the posting when Accounting is down and may double-post on retry; the outbox is committed with the business data so nothing is lost	Alternatives: direct call, message broker. Consequence: the ledger trails stock by a few seconds (eventual consistency)
ADR-02	Money as BigInt satang, sent as strings in JSON	Floats round incorrectly and JSON numbers are only safe up to 2^53	Consequence: clients parse strings; Excel falls back to text beyond 2^53
ADR-03	Enforce posted-entry immutability with database triggers as well	Application checks alone can be bypassed by scripts or direct SQL	Consequence: rules live in a hand-written migration that must be kept in step with the code
ADR-04	Credit GR/IR (2150) instead of trade payables (2100) on receipt	No invoice exists at receipt time and there are no payments yet, so payables would only ever grow	Consequence: adding AP invoices later is one new rule, Dr 2150 / Cr 2100, without touching posted entries
ADR-05	Number entries with a row-locked monthly counter (INSERT … ON CONFLICT DO UPDATE … RETURNING)	PostgreSQL sequences can skip numbers on rollback, which is unacceptable for accounting documents	Consequence: posts within one month serialize, which is acceptable at this volume

ID	Decision	Rationale	Alternatives / consequences
ADR-06	System events in a closed period move to the next open period instead of being rejected	Rejecting would leave the outbox stuck forever and the ledger out of step with stock	Consequence: the booking date may differ from the actual date, which is therefore recorded in the description and memo
ADR-07	Keep business rules in pure functions	Fast, exhaustive tests without a database	Consequence: services shrink to loading, locking and saving

5. Detailed design

5.1 Posting procedure

postEntryTx runs in one transaction, in this order:

- Lock the entry row with SELECT … FOR UPDATE
- Check that the status may move to POSTED (status machine)
- Re-check balance and that accounts are still postable and active
- Find the Bangkok period of the entry date, create it if missing, reject if closed (409)
- Increment the month counter under a row lock and build JE-YYYY-MM-NNNN
- Save status POSTED, number, poster and time

5.2 Idempotent event booking

- Validate with Zod (discriminated union on topic, version = 1); zero or negative amounts give 400
- Translate the event into planned lines by account code (pure function)
- If an entry with the same (sourceModule, sourceType, sourceId) exists, return 200 with that entry
- Otherwise resolve accounts by code (422 if missing), resolve an open period, create and post the draft in one transaction, and return 201
- If identical requests race and hit the unique key (Prisma P2002), return the winning entry

5.3 Year-end close

The plan is computed once and shared by preview and close, so they cannot disagree. Blockers are: the year is already closed, an earlier year is open, drafts are dated in the year, or retained earnings is missing or inactive. When clear, the system books an entry on 31 December (noon Bangkok time) that brings every revenue and expense account to zero, puts the difference in 3200 (credit for profit, debit for loss), and locks all 12 periods, all in one transaction.

5.4 Report computation

- Totals are aggregated in the database with SUM(…)::bigint rather than summed in application code
- Trial balance: each account's net balance goes to the debit or credit side, then the two totals are compared
- Balance sheet: assets = liabilities + equity + unclosed net income, so it balances at any date
- Date boundaries: the end date includes the whole Bangkok day

5.5 Error model

Status	When
400	Invalid input (Zod), bad date or period format, end before start

Status	When
401	Missing or invalid token, missing or wrong service key
403	No module access or missing role
404	Entry or account not found
409	State conflict: closed period, invalid transition, editing a used account, double reversal, year-end blockers
422	Accounting rule broken: unbalanced, both or neither side, group or inactive account, mapped account missing
500	Unexpected error, logged on the server

5.6 Configuration

Variable	Meaning
JWT_SECRET	Must equal K-ERPHub's secret
ACCOUNTING_SERVICE_KEY	Shared with Stock's outbox worker; never committed or logged
DATABASE_URL	PostgreSQL connection string
PORT	API port (4002)
FRONTEND_DIST (optional)	Location of the built SPA

6. Test documentation

6.1 Test plan

Item	Detail
Objective	Show that every accounting and integrity rule in Section 3 holds, including those enforced by the database
Test levels	Unit (pure functions), integration (real HTTP + real PostgreSQL), end-to-end across two modules
Tools	Vitest 2, Node.js fetch, e2e script, ESLint, oxlint
Environment	Separate <name>_test database, created and migrated automatically; the run refuses any non-test database; integration files run one at a time
Entry criteria	Packages installed; PostgreSQL running (for integration/e2e)
Exit criteria	All tests pass, trial balance balances after every scenario, no lint errors

6.2 Test design

Suite	Cases	Features tested
journal.validator.test.ts	15	Balance, one side, negatives, above int32, reversal nets to zero, status machine
posting-rules.test.ts	9	Each event mapping, movement-id key, rejecting zero, negative, unknown topic and version
statements.test.ts	11	Income statement, balance sheet, closing plan (profit, loss, break-even)
period.test.ts	6	Bangkok month boundaries
journal.integration.test.ts	26	Gap-free numbering under concurrency, closed periods, reversal, CHECK and triggers versus raw SQL, JWT, module and role checks
postings.integration.test.ts	16	Service-key isolation, idempotency (5 concurrent events give 1 entry), Bangkok time, period shifting, 1300 reconciliation
statements.integration.test.ts	18	Statements, year-end close (blockers, order, loss, locked periods), Excel export
stock-accounting.e2e.mjs	23	Cross-module outbox delivery, Accounting down then back, permanent failure and manual retry, opening balance, reconciliation

6.3 Test completion report

Executed on 12 October 2026 on a copy of the source (excluding .env) in an isolated workspace, after a fresh npm ci.

Item	Result	Detail
Unit tests (Vitest)	Passed	41/41 passed in 4 files
ESLint (backend)	Passed	No problems

Item	Result	Detail
oxlint (frontend)	Passed	0 errors, 3 warnings (see AN-02)
Frontend build	Passed	41 modules, 292 kB JS (90 kB gzip)
Integration tests (60)	Not executed	See AN-01
End-to-end (23 checks)	Not executed	See AN-01

6.4 Incident (anomaly) report

ID	Description	Severity	Status / action
AN-01	The test environment could not download the Prisma engine (HTTP 403), so the Prisma Client could not be generated and integration/e2e could not run. This is an environment limitation, not a product defect.	High (evidence)	Open: run npm test and the e2e script on the development machine and record the results in 6.3
AN-02	oxlint warnings: me.tsx exports a component and a hook (fast refresh); PeriodsPage calls new Date() during render; JournalEntryFormPage calls setState inside useEffect	Low	Open: code tidy-up, no functional impact

7. Quality evaluation (ISO/IEC 25010)

Based on reading the source and the test results in Section 6. Ratings: Good = clear design and test evidence; Adequate = supported by design, evidence incomplete; Needs improvement = an identified gap; Not measured = no data yet.

Characteristic	Evidence	Rating
Functional suitability	FR-01 to FR-21 implemented; accounting rules covered by tests; BigInt throughout	Good
Performance efficiency	SQL aggregation with suitable indexes; no load test yet; entry list capped at 500 rows without pagination	Not measured
Compatibility	REST/JSON, versioned events, retryable versus permanent responses	Good
Interaction capability	Live balance in the form, buttons hidden by role, clear error messages; no user testing yet	Adequate
Reliability	Outbox, idempotency and backoff retries, row locks under concurrency; tests designed but not executed in this assessment	Adequate
Security	JWT, RBAC and module access, constant-time service key, immutability triggers; but CORS allows any origin and all modules share one secret	Adequate
Maintainability	Clear layers, pure core, single account mapping, 41 unit + 60 integration + 23 e2e tests	Good
Flexibility	Configured by .env, installed with Docker and npm; no Dockerfile for the application yet	Adequate
Safety	Not directly applicable to an accounting system	—

7.1 Findings and recommendations

ID	Finding	Recommendation
F-01	app.ts uses cors() with defaults, which allows any origin	Restrict to the K-ERPHub origin, or disable CORS since calls go through the proxy
F-02	All modules share one HS256 JWT_SECRET; a leak in any module allows forging hub tokens	Move to RS256/EdDSA: the hub signs with a private key, modules verify with the public key
F-03	GET /api/journal-entries returns at most 500 rows with no pagination	Add cursor-based pagination
F-04	The outbox worker supports a single instance (guarded by an in-process flag)	Use FOR UPDATE SKIP LOCKED before running several instances
F-05	Stock still stores amounts as Int; POs above about 21.4 million THB would overflow on the Stock side	Migrate Stock to BigInt to match Accounting
F-06	No integration or e2e results in this assessment (AN-01)	Run locally and set up CI (e.g. GitHub Actions with a PostgreSQL service)
F-07	No load test or response-time measurement yet	Measure trial balance and statements with large synthetic data

8. Requirements traceability matrix

Req.	Design element / code	Verified by	Status
FR-01	account.service.ts (assertParentOk)	journal.integration: chart of accounts rules	Done
FR-02	account.service.ts (updateAccount)	journal.integration: blocks changing code/type…	Done
FR-03	account.service.ts (deleteAccount)	journal.integration: lets an unused account be edited and deleted	Done
FR-04	journal.service.ts (createDraft, updateDraft, deleteDraft)	journal.integration: only DRAFT entries can be edited or deleted	Done
FR-05	journal.validator.ts (assertBalanced)	journal.validator.test (9 cases); journal.integration: rejects an unbalanced draft	Done
FR-06	journal.service.ts (nextEntryNo)	journal.integration: gap-free numbers; concurrent posts	Done
FR-07	journal.service.ts (reverseEntry), buildReversalLines	journal.validator.test; journal.integration: reversal (4 cases)	Done
FR-08	period.service.ts (ensurePeriod), utils/period.ts	period.test; journal.integration: uses Bangkok time	Done
FR-09	period.service.ts (closePeriod, assertPeriodOpen)	journal.integration: closed period 409; only Admin may close	Done
FR-10	year-end.ts, year-end.service.ts	statements.test: buildClosingPlan; statements.integration: year-end close (10 cases)	Done
FR-11	reporting.service.ts (trialBalance)	journal.integration: reports	Done
FR-12	reporting.service.ts (generalLedger)	journal.integration: general ledger running balance	Done
FR-13	statements.ts (buildIncomeStatement), accountTotals(excludeClosing)	statements.test; statements.integration: income statement	Done
FR-14	statements.ts (buildBalanceSheet)	statements.test; statements.integration: balance sheet	Done
FR-15	excel.ts, reporting.routes.ts (*.xlsx)	statements.integration: Excel export (3 cases)	Done
FR-16	posting-rules.ts, mapping.ts	posting-rules.test; postings.integration: GOODS_RECEIVED, STOCK_ADJUSTED	Done
FR-17	postings.service.ts (postEvent), unique source key	postings.integration: idempotent; 5 concurrent events	Done
FR-18	period.service.ts (resolveEventDate)	postings.integration: closed period moves to next open	Done
FR-19	auth.middleware.ts, service-key.middleware.ts	journal.integration: auth (5 cases); postings.integration: service key	Done
FR-20	auth.routes.ts (GET /api/me), lib/me.tsx	journal.integration: /api/me returns the caller's roles	Done
FR-21	journal.service.ts (listEntries)	Demonstrated in the Journal Entries page	Done

Req.	Design element / code	Verified by	Status
NFR-01	BigInt columns, SUM()::bigint, JSON replacer	Above-int32 tests (unit + integration)	Done
NFR-02	Stock outbox + delivery.ts	e2e: Accounting down, then delivered	Done
NFR-03	migration: CHECK + triggers	journal.integration: database-level integrity	Done
NFR-06	row locks, counter upsert, P2002 fallback	journal.integration: concurrent posts; postings.integration: concurrency	Done

9. Limitations, risks and future work

9.1 Intentional limitations

- No tax, AP invoices or payments, receivables, multi-currency or multi-company
- Inventory is valued at PO unit cost and current product cost, not FIFO or average
- Failed outbox rows are visible through Stock's API only, with no web screen yet

9.2 Risks

Risk	Level	Mitigation
JWT_SECRET leaks from any module	High	Asymmetric signing (F-02)
Skipping the opening balance at cut-over leaves 1300 below real stock	Medium	OPENING_BALANCE event and reconciliation test
Large POs overflow Int on the Stock side	Low	Migrate Stock to BigInt (F-05)
Several workers pick the same row	Low	Accounting is idempotent already; add SKIP LOCKED (F-04)

9.3 Future work

- Supplier invoices and payments (clear GR/IR with Dr 2150 / Cr 2100, then Dr 2100 / Cr 1110)
- VAT 7% and withholding tax
- CI with PostgreSQL, and Dockerfiles for deployment
- Resolve findings F-01 to F-07

10. Conclusion

The K-ERP Accounting module is a double-entry general ledger that meets all 21 functional requirements. Its strengths are two-layer integrity (code and database), BigInt money throughout, gap-free numbering under concurrency, and exactly-once cross-service booking through a transactional outbox. All unit tests pass. To be production-ready, the next steps are to run and record the integration and e2e results, set up CI, and resolve security findings F-01 and F-02.

Appendix A. API reference

Method	Path	Role	Description
GET	`/health`	None	Health check
GET	`/api/me`	Member	username, roles
GET	`/api/accounts`	Member	All accounts by code
GET	`/api/accounts/:id`	Member	One account
POST	`/api/accounts`	Accountant/ Admin	Create account
PATCH	`/api/accounts/:id`	Accountant/ Admin	Update account
DELETE	`/api/accounts/:id`	Accountant/ Admin	Delete unused account
GET	`/api/journal-entries`	Member	status, from, to, origin
GET	`/api/journal-entries/:id`	Member	Entry detail
POST	`/api/journal-entries`	Accountant/ Admin	Create draft
PATCH	`/api/journal-entries/:id`	Accountant/ Admin	Replace draft
DELETE	`/api/journal-entries/:id`	Accountant/ Admin	Delete draft
POST	`/api/journal-entries/:id/post`	Accountant/ Admin	Post
POST	`/api/journal-entries/:id/reverse`	Accountant/ Admin	Reverse
GET	`/api/periods`	Member	Periods
POST	`/api/periods/:period/close`	Admin	Close period
GET	`/api/year-end/:year/preview`	Admin	Preview year-end
POST	`/api/year-end/:year/close`	Admin	Close year
GET	`/api/reports/trial-balance`	Member	asOf
GET	`/api/reports/general-ledger/:accountId`	Member	from, to
GET	`/api/reports/income-statement`	Member	from, to
GET	`/api/reports/balance-sheet`	Member	asOf
GET	`/api/reports/{trial-balance|income-statement|balance-sheet}.xlsx`	Member	Excel download
POST	`/api/postings`	X-Service-Key	Stock events (201 new / 200 replay)

Appendix B. Data dictionary

accounts

Column	Type	Key	Description
id	TEXT	PK	cuid
code	TEXT	UK	3–10 digits
name	TEXT		Account name
type	AccountType		ASSET | LIABILITY | EQUITY | REVENUE | EXPENSE
parentId	TEXT	FK	Parent group account
isPostable	BOOLEAN		false = group header
isActive	BOOLEAN		Active

journal_entries

Column	Type	Key	Description
id	TEXT	PK	cuid
entryNo	TEXT	UK	JE-YYYY-MM-NNNN, null while DRAFT
entryDate	TIMESTAMP		Accounting date, decides the period
description	TEXT		Description
status	JournalStatus		DRAFT | POSTED | REVERSED
sourceModule / sourceType / sourceId	TEXT	UK	Origin and idempotency key
sourceRef	JSONB		Link back to the source document
reversalOfId	TEXT	FK, UK	Entry being reversed
createdBy / postedBy	TEXT		JWT sub or system:stock
postedAt / createdAt	TIMESTAMP		Timestamps

journal_lines

Column	Type	Key	Description
id	TEXT	PK	cuid
entryId	TEXT	FK	journal_entries (cascade)
accountId	TEXT	FK	accounts (restrict)
debit	BIGINT		Satang
credit	BIGINT		Satang (CHECK one_side_only)
memo	TEXT		Memo

accounting_periods

Column	Type	Key	Description
id	TEXT	PK	uuid
period	TEXT	UK	YYYY-MM
status	PeriodStatus		OPEN | CLOSED
closedBy / closedAt	TEXT / TIMESTAMP		Closed by and when

document_counters

Column	Type	Key	Description
key	TEXT	PK	JE-YYYY-MM
lastNo	INTEGER		Last number issued

Appendix C. Default chart of accounts

Code	Name	Type	Used by
1000	Assets (group)	ASSET	
1100	Cash	ASSET	
1110	Bank deposits	ASSET	
1200	Trade receivables	ASSET	
1300	Inventory	ASSET	Receipts, adjustments, opening
2000	Liabilities (group)	LIABILITY	
2100	Trade payables	LIABILITY	
2150	Goods received not invoiced (GR/IR)	LIABILITY	Receipts
3000	Equity (group)	EQUITY	
3100	Share capital	EQUITY	
3200	Retained earnings	EQUITY	Year-end, opening
4000	Revenue (group)	REVENUE	
4100	Sales revenue	REVENUE	
5000	Expenses (group)	EXPENSE	
5100	Cost of goods sold	EXPENSE	
5200	Stock adjustment loss	EXPENSE	Adjustments
5300	Operating expenses	EXPENSE	

Appendix D. Automatic posting rules

Event	Debit	Credit	Idempotency key (sourceId)
GOODS_RECEIVED (goods received against a PO)	1300 Inventory	2150 GR/IR	Goods receipt id
STOCK_ADJUSTED (decrease)	5200 Adjustment loss	1300 Inventory	Stock movement id
STOCK_ADJUSTED (increase)	1300 Inventory	5200 Adjustment loss	Stock movement id
OPENING_BALANCE (cut-over)	1300 Inventory	3200 Retained earnings	Cut-over date
Year-end close (internal)	Revenue / 3200 if loss	Expenses / 3200 if profit	Year (accounting, YEAR_END_CLOSE)